

Data Processing

Stony Creek, Tivoli North, and Field Station

Ilianna Ditren, Joy Terentiev, Natalie Novoseltseva

Background Information

This document contains the processing steps to download the main datasets of this exploratory data analysis. The first dataset contains water quality data from Stony Creek, the following contains water quality data from Tivoli North, and the third and final dataset contains water quality data from Field Station. Water quality on Stony Creek was explored first, and then the weather data from Field Station. Tivoli North water quality data was not explored, although the same steps taken to explore Stony Creek can be taken after an appropriate workflow is established. After exploring data on Stony Creek and data on Field Station weather individually, those datasets were combined in order to analyze them together.

Each dataset was downloaded directly from the web. There is a separate file for each year. Those files are placed in a folder and then read into a single csv file. Once the data is in the csv file, the data is cleaned by removing empty rows and columns, and deleted flagged data. Finally, the data is summarized by taken the mean weekly values and the mean daily values.

Ultimately, there are 5 main datasets:

- **stony_creek**: Water quality data from Stony Creek recorded every 15 minutes
- **fs_weather_01_26**: Weather data from Field Station recorded every 15 minutes
- **combined**: Water quality from the **stony_creek** dataset and weather data from the **fs_weather_01_26** dataset in a single dataset
- **daily_combined**: Water quality from the **stony_creek** dataset and weather data from the **fs_weather_01_26** dataset in a single dataset summarized by daily averages
- **weekly_combined**: Water quality from the **stony_creek** dataset and weather data from the **fs_weather_01_26** dataset in a single dataset summarized by weekly averages

Project Set Up

Downloading libraries

```
library(readr)
library(tidyverse)
library(dplyr)
library(lubridate)
library(janitor)
library(stringr)
library(reshape2)
```

Data and Cleaning

To obtain raw water quality and weather data, use the Centralized Data Management Office (CDMO) website: <https://cdmo.baruch.sc.edu/>

1. Navigate to the CDMO website: <https://cdmo.baruch.sc.edu/>.
2. Click on the **Get Data** tab at the top of the page.
3. Select **Advanced Query System** and click **Launch**.
4. Locate the **Zip Downloads** option and click **Choose ZIP Files**.
5. From the list of National Estuarine Research Reserves, select **Hudson River, NY**.
6. Select the appropriate dataset:
 - For **weather data** from the field station, select **hudfsmet-p**.
 - For **Stony Creek water quality data**, select **hudscwq-p**.
 - For **Tivoli North water quality data**, select **hudtnwq-p**.

Each dataset must be requested separately, as the website generates one download link per request, and the analysis code is structured to process each dataset independently.

7. After selecting the desired site, click **Submit location and proceed to next step**.
8. Select the desired range of years and click **Get files**.
 - For the DA Capstone Spring 2026 project, requests were made for all available years.
9. Complete the required information requested by the website and submit the request.

10. Shortly after submission, an email will be sent to the provided email address containing a download link to the data files.
11. Copy and paste this link into the corresponding section of the code for the selected site.
12. Run the code. After execution, all files will be downloaded into a subfolder named after the site within your data folder.

Note: Download link expires after certain amount of time. Ensure to download the data soon after you receive the link.

Stony Creek Water Quality (2002-2021)

Data Download

```
#creates a folder for the new data
dir.create("data/stony_creek", recursive = TRUE, showWarnings = FALSE)

## downloading Stony Creek data file from the web

#creating a temporary zip file
zip_sc <- tempfile(fileext = ".zip")

link_stony_creek <- "http://cdmo.baruch.sc.edu/aqs/output/334470.zip"

download.file(link_stony_creek, zip_sc, mode = "wb")

#unzipping data from zipfile and placing it in a folder
unzip(zip_sc, exdir = "data/stony_creek")
```

```
#read in the files for all the years as a one data file using regex (regular expressions)
files <- list.files(
  "data/stony_creek",
  pattern = "^hudscwq[0-9]{4}\\\\.csv$", # ^ -starts a string, hudscwq - the exact name in the
  full.names = TRUE
)

# read files matcheing the pattern
stony_creek_raw <- do.call(
  rbind,
  lapply(files, function(f) {
```

```

df <- read.csv(f, fileEncoding = "Latin1")
df$year <- as.integer(gsub(".*([0-9]{4})\\.csv$", "\\1", f))
df
})
)

```

Data Cleaning

```

flagged_data <- "<-3>|<-2>|<1>"

flags <- c("F_Record",
          "F_Temp",
          "F_SpCond",
          "F_Sal",
          "F_DO_Pct",
          "F_Depth",
          "F_cDepth",
          "F_Level",
          "F_pH",
          "F_Turb",
          "F_Chlor",
          "F_DO_mgl")

stony_creek <- stony_creek_raw |>
  mutate(Temp = if_else(str_detect(F_Temp, flagged_data) & year != 2019, NA_real_, Temp),
         SpCond = if_else(str_detect(F_SpCond, flagged_data) & year != 2019, NA_real_, SpCond),
         Sal = if_else(str_detect(F_Sal, flagged_data) & year != 2019, NA_real_, Sal),
         DO_Pct = if_else(str_detect(F_DO_Pct, flagged_data) & year != 2019, NA_real_, DO_Pct),
         Depth = if_else(str_detect(F_Depth, flagged_data) & year != 2019, NA_real_, Depth),
         cDepth = if_else(str_detect(F_cDepth, flagged_data) & year != 2019, NA_real_, cDepth),
         pH = if_else(str_detect(F_pH, flagged_data) & year != 2019, NA_real_, pH),
         Turb = if_else(str_detect(F_Turb, flagged_data) & year != 2019, NA_real_, Turb),
         Chlor = if_else(str_detect(F_Chlor, flagged_data) & year != 2019, NA_real_, Chlor),
         DO_mgl = if_else(str_detect(F_DO_mgl, flagged_data) & year != 2019, NA_real_, DO_mgl)
  )

# Removing empty columns and other irrelevant columns
stony_creek <- stony_creek |>
  remove_empty("cols")|>

```

```

select(-StationCode,
      -isSWMP, #historical and provisionalPlus columns indicate the status of QAQC
      -Historical,
      -ProvisionalPlus
    )

#removing flag columns
stony_creek <- stony_creek %>% select(-all_of(flags))

```

Cleaning Date and Time Columns

```

#separate the date and time column
stony_creek <- stony_creek %>%
  mutate(
    DateTimeStamp = as.POSIXct(DateTimeStamp, format = "%m/%d/%Y %H:%M"), #converts the vari
    date = as.Date(DateTimeStamp),
    time = format(DateTimeStamp, "%H:%M:%S")
  )

#create a month column
stony_creek <- stony_creek %>%
  mutate(
    month = month(DateTimeStamp)
  )

#create a season column
stony_creek <- stony_creek %>%
  mutate(
    season = case_when(
      month %in% c(12, 1, 2) ~ "Winter",
      month %in% 3:5 ~ "Spring",
      month %in% 6:8 ~ "Summer",
      month %in% 9:11 ~ "Fall"
    )
  )

```

```

#the code to write out the dataset
write_csv(stony_creek, "data/stony_creek.csv")

```

```

#uncomment the code to write out the new/updated version of the dataset. It might be better t

```

Tivoli North Water Quality (1996 - 2025)

Data Download

```
#creates a folder for the new data
dir.create("data/tivoli_north", recursive = TRUE, showWarnings = FALSE)

#creating a temporary zip file
zip_tn <- tempfile(fileext = ".zip")

tiv_north_raw <- "http://cdmo.baruch.sc.edu/aqs/output/334470.zip"

download.file(tiv_north_raw, zip_tn, mode = "wb")

#unzipping data from and placing it the created folder
unzip(zip_tn, exdir = "data/tivoli_north")
```

```
#read in the files for all the years as a one data file
files <- list.files(
  "data/tivoli_north",
  pattern = "^hudtnwq[0-9]{4}\\\\.csv$", # ^ -starts a string, hudtnwq - the exact name in the
  full.names = TRUE
)

tn_96_25 <- do.call(
  rbind,
  lapply(files, function(f) {
    df <- read.csv(f, fileEncoding = "Latin1")
    df$year <- as.integer(gsub(".*([0-9]{4})\\.csv$", "\\1", f))
    df
  })
)
```

Field Station Meteorological Data (2001-2026)

Data Download

```

#creates a data folder
dir.create("data/fs_weather_01_26", recursive = TRUE, showWarnings = FALSE)

#using tempfile() to create a tempoary storage
zip <- tempfile(fileext = ".zip")

#downlod the file from the url
download.file("http://cdmo.baruch.sc.edu/aqs/output/334470.zip", zip, mode = "wb")

#unzipping data from and placing it the created folder
unzip(zip, exdir = "Spring2026/data/fs_weather_01_26")

#read in the files for all the years as a one data file hudfsmet-p
files <- list.files(
  "Spring2026/data/fs_weather_01_26",
  pattern = "^hudfsmet[0-9]{4}\\..csv$", # ^ -starts a string
                                          # hudfsmet - the exact name in the file (constant)
                                          # any digits from 0-9 might follow
                                          # {4} - we are looking for 4 digits pattern
                                          # \\.. - there should be a dot
                                          # csv - it should end with "csv"
                                          # end a string
  full.names = TRUE
)

fs_weather_01_26_raw <- do.call(
  rbind,
  lapply(files, function(f) {
    df <- read.csv(f, fileEncoding = "Latin1")
    df$year <- as.integer(gsub(".*([0-9]{4})\\..csv$", "\\1", f))
    df
  })
)

```

Data Cleaning

```

# Removing empty columns and other irrelevant columns
fs_weather_01_26 <- fs_weather_01_26_raw |>
  remove_empty("cols")|>
  select(-StationCode,

```

```

        -isSWMP, #historical and provisional Plus columns indicate the status of QAQC
        -Historical,
        -ProvisionalPlus,
    )

#creating a vector with all the weather flags
flags_weather <- c(
  "F_Record",
  "F_ATemp",
  "F_RH",
  "F_BP",
  "F_WSpd",
  "F_MaxWSpd",
  "F_Wdir",
  "F_SDWDir",
  "F_TotPAR",
  "F_TotPrcp",
  "F_TotSoRad")

#creating a vector with all the weather flags that are to be removed from the dataset

fs_weather_01_26 <- fs_weather_01_26 |>
  mutate(ATemp = if_else(str_detect(F_ATemp, flagged_data), NA_real_, ATemp),
         RH = if_else(str_detect(F_RH, flagged_data), NA_real_, RH),
         BP = if_else(str_detect(F_BP, flagged_data), NA_real_, BP),
         WSpd = if_else(str_detect(F_WSpd, flagged_data), NA_real_, WSpd),
         MaxWSpd = if_else(str_detect(F_MaxWSpd, flagged_data), NA_real_, MaxWSpd),
         Wdir = if_else(str_detect(F_Wdir, flagged_data), NA_real_, Wdir),
         SDWDir = if_else(str_detect(F_SDWDir, flagged_data), NA_real_, SDWDir),
         TotPAR = if_else(str_detect(F_TotPAR, flagged_data), NA_real_, TotPAR),
         TotPrcp = if_else(str_detect(F_TotPrcp, flagged_data), NA_real_, TotPrcp)
  )

#removing flag columns
fs_weather_01_26 <- fs_weather_01_26 |>
  select(-all_of(flags_weather))

#separate the data and time column
fs_weather_01_26 <- fs_weather_01_26 %>%
  mutate(
    DatetimeStamp = as.POSIXct(DatetimeStamp, format = "%m/%d/%Y %H:%M"), #converts the vari
    date = as.Date(DatetimeStamp),

```



```

    time = format(Datetimestamp, "%H:%M:%S")
  )

#create a month column
fs_weather_01_26 <- fs_weather_01_26 %>%
  mutate(
    month = month(Datetimestamp)
  )

#create a season column
fs_weather_01_26 <- fs_weather_01_26 %>%
  mutate(
    season = case_when(
      month %in% c(12, 1, 2) ~ "Winter",
      month %in% 3:5 ~ "Spring",
      month %in% 6:8 ~ "Summer",
      month %in% 9:11 ~ "Fall"
    )
  )

```

```

#the code to write out the dataset

```

```

write_csv(fs_weather_01_26, "data/fs_weather_01_26.csv")

```

```

#uncomment the code to write out the new/updated version of the dataset.

```

Joining Water Quality and Weather Data

```

# reading main datasets
water <- read_csv("data/stony_creek.csv")
weather <- read_csv("data/fs_weather_01_26.csv")

```

Exploring the structure of the data

```

str(weather)

```

```

spc_tbl_ [889,642 x 17] (S3: spec_tbl_df/tbl_df/tbl/data.frame)

```

```

$ DateTimeStamp: POSIXct[1:889642], format: "2001-01-01 05:15:00" "2001-01-01 05:30:00" ...
$ Frequency      : num [1:889642] 15 15 15 15 15 15 15 15 15 15 ...
$ ATemp          : num [1:889642] NA NA NA NA NA NA NA NA NA NA ...
$ RH             : num [1:889642] NA NA NA NA NA NA NA NA NA NA ...
$ BP             : num [1:889642] NA NA NA NA NA NA NA NA NA NA ...
$ WSpd           : num [1:889642] NA NA NA NA NA NA NA NA NA NA ...
$ MaxWSpd        : num [1:889642] NA NA NA NA NA NA NA NA NA NA ...
$ MaxWSpdT       : 'hms' num [1:889642] NA NA NA NA ...
..- attr(*, "units")= chr "secs"
$ Wdir           : num [1:889642] NA NA NA NA NA NA NA NA NA NA ...
$ SDWDir         : num [1:889642] NA NA NA NA NA NA NA NA NA NA ...
$ TotPAR         : num [1:889642] NA NA NA NA NA NA NA NA NA NA ...
$ TotPrpcp       : num [1:889642] NA NA NA NA NA NA NA NA NA NA ...
$ year           : num [1:889642] 2001 2001 2001 2001 2001 ...
$ date           : Date[1:889642], format: "2001-01-01" "2001-01-01" ...
$ time           : 'hms' num [1:889642] 00:15:00 00:30:00 00:45:00 01:00:00 ...
..- attr(*, "units")= chr "secs"
$ month          : num [1:889642] 1 1 1 1 1 1 1 1 1 1 ...
$ season         : chr [1:889642] "Winter" "Winter" "Winter" "Winter" ...
- attr(*, "spec")=
.. cols(
..   DateTimeStamp = col_datetime(format = ""),
..   Frequency = col_double(),
..   ATemp = col_double(),
..   RH = col_double(),
..   BP = col_double(),
..   WSpd = col_double(),
..   MaxWSpd = col_double(),
..   MaxWSpdT = col_time(format = ""),
..   Wdir = col_double(),
..   SDWDir = col_double(),
..   TotPAR = col_double(),
..   TotPrpcp = col_double(),
..   year = col_double(),
..   date = col_date(format = ""),
..   time = col_time(format = ""),
..   month = col_double(),
..   season = col_character()
.. )
- attr(*, "problems")=<externalptr>

```

```
str(water)
```

```
spc_tbl_ [592,276 x 16] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ DateTimeStamp: POSIXct[1:592276], format: "2002-04-08 15:30:00" "2002-04-08 16:00:00" ...
 $ Temp         : num [1:592276] 7 7.1 7.2 7.3 7.5 7.7 7.8 8.1 8.3 8.5 ...
 $ SpCond       : num [1:592276] 0.33 0.33 0.33 0.33 0.33 0.33 0.33 0.32 0.32 0.32 0.32 ...
 $ Sal          : num [1:592276] 0.2 0.2 0.2 0.2 0.2 0.2 0.2 0.2 0.2 0.2 0.2 ...
 $ DO_Pct       : num [1:592276] 106 106 107 108 109 ...
 $ DO_mgl       : num [1:592276] 12.8 12.9 12.9 13 13.1 13.1 13 12.9 12.9 12.7 ...
 $ Depth        : num [1:592276] 0.14 0.14 0.13 0.13 0.12 0.12 0.12 0.12 0.11 0.11 ...
 $ cDepth       : num [1:592276] NA NA NA NA NA NA NA NA NA NA ...
 $ pH           : num [1:592276] 8 8 8.1 8.1 8.2 8.2 8.2 8.2 8.2 8.2 ...
 $ Turb         : num [1:592276] 4 4 4 4 4 4 4 4 4 4 ...
 $ ChlFluor     : num [1:592276] NA NA NA NA NA NA NA NA NA NA ...
 $ year         : num [1:592276] 2002 2002 2002 2002 2002 ...
 $ date         : Date[1:592276], format: "2002-04-08" "2002-04-08" ...
 $ time         : 'hms' num [1:592276] 11:30:00 12:00:00 12:30:00 13:00:00 ...
 ..- attr(*, "units")= chr "secs"
 $ month        : num [1:592276] 4 4 4 4 4 4 4 4 4 4 ...
 $ season       : chr [1:592276] "Spring" "Spring" "Spring" "Spring" ...
 - attr(*, "spec")=
 .. cols(
 ..   DateTimeStamp = col_datetime(format = ""),
 ..   Temp = col_double(),
 ..   SpCond = col_double(),
 ..   Sal = col_double(),
 ..   DO_Pct = col_double(),
 ..   DO_mgl = col_double(),
 ..   Depth = col_double(),
 ..   cDepth = col_double(),
 ..   pH = col_double(),
 ..   Turb = col_double(),
 ..   ChlFluor = col_double(),
 ..   year = col_double(),
 ..   date = col_date(format = ""),
 ..   time = col_time(format = ""),
 ..   month = col_double(),
 ..   season = col_character()
 .. )
 - attr(*, "problems")=<externalptr>
```

```
names(weather)
```

```
[1] "DateTimeStamp" "Frequency"      "ATemp"          "RH"  
[5] "BP"            "WSpd"           "MaxWSpd"        "MaxWSpdT"  
[9] "Wdir"          "SDWDir"         "TotPAR"         "TotPrcp"  
[13] "year"          "date"           "time"           "month"  
[17] "season"
```

```
names(water)
```

```
[1] "DateTimeStamp" "Temp"           "SpCond"         "Sal"  
[5] "DO_Pct"        "DO_mgl"         "Depth"          "cDepth"  
[9] "pH"           "Turb"           "ChlFluor"       "year"  
[13] "date"         "time"           "month"          "season"
```

```
#rename the column in the water dataset to easier join the datasets  
weather <- weather %>%  
  rename(DateTimeStamp = DateTimeStamp)
```

Joining the water quality and weather datasets

```
#join water and weather data with inner join  
combined <- inner_join(water, weather, by = "DateTimeStamp")
```

```
#get rid of the duplicate columns and  
combined <- combined %>%  
  select(-year.y, -date.y, -time.y, -month.y, -season.y) %>%  
  rename(  
    year = year.x,  
    date = date.x,  
    time = time.x,  
    month = month.x,  
    season = season.x  
  )
```

```
#the code to write out the dataset  
write_csv(combined, "data/combined.csv")
```

The combined dataset contained variables measured every 15-30 minutes. In this section, the data will be summarized to contain daily averages only.

```
# computing the daily mean values for all variables in the combined_names object
daily_combined <- combined |>
  group_by(date) |>
  summarize(across(
    where(is.numeric), ~ mean(.x, na.rm = TRUE)), .groups = "drop") |>
  mutate(
    season_astro = case_when(
      date >= as.Date(paste0(year, "-12-21")) | date < as.Date(paste0(year, "-03-20")) ~ "Winter",
      date >= as.Date(paste0(year, "-03-20")) & date < as.Date(paste0(year, "-06-21")) ~ "Spring",
      date >= as.Date(paste0(year, "-06-21")) & date < as.Date(paste0(year, "-09-22")) ~ "Summer",
      TRUE ~ "Fall"
    )
  )

#writing new csv
write_csv(daily_combined, "data/daily_combined.csv")
```

```
# recode the seasons:

combined <- combined %>%
  mutate(
    season_astro = case_when(
      date >= as.Date(paste0(year, "-12-21")) | date < as.Date(paste0(year, "-03-20")) ~ "Winter",
      date >= as.Date(paste0(year, "-03-20")) & date < as.Date(paste0(year, "-06-21")) ~ "Spring",
      date >= as.Date(paste0(year, "-06-21")) & date < as.Date(paste0(year, "-09-22")) ~ "Summer",
      TRUE ~ "Fall"
    )
  )

# calculate weekly averages:
weekly_combined <- combined %>% #create a new dataset
  mutate(week = floor_date(date, unit = "week")) %>% #create a new "week" column
  group_by(week) %>% #group by week
  summarise(
    across(
      where(is.numeric), #apply only numeric columns
      ~ mean(.x, na.rm = TRUE) #calculate the mean
    ),
    .groups = "drop"
```

```
)  
  
#writing new csv  
write_csv(weekly_combined, "data/weekly_combined.csv")
```